Kasi Yedumati · Full Stack Training

Lesson 5 — Database Connectivity

Course: FastAPI for Full Stack · **Session 5 of 10** · Learning path · **Previous:** Lesson 4 · **Next:** Lesson 6

> **Module Outcome:** You can create an `employee` table in `training_db`, insert the six AcmeHR people, and read them back through FastAPI.

You will build	`GET /employees` reading MySQL instead of a Python list
New words	Engine, session, table model
You need	MySQL running, database `training_db`, and `EmployeeOut`

From the requirements. Employee records are the source of truth for the directory and the profile. A list inside `main.py` disappears when `fastapi dev` stops. The rows belong in MySQL.

The practice login is the one from the MySQL setup lesson: user `trainee`, password `password123`, database `training_db`. If your password differs, change `DATABASE_URL`.

---

1. The table, then the Python class



Run this once in MySQL. The Python code also creates the table on startup. Seeing the SQL first tells you what the class means.

```
USE training_db;

CREATE TABLE IF NOT EXISTS employee (
  id INT PRIMARY KEY AUTO_INCREMENT,
  employee_id VARCHAR(10) NOT NULL UNIQUE,
  name VARCHAR(80) NOT NULL,
  designation VARCHAR(80) NOT NULL,
  department VARCHAR(80) NOT NULL,
  email VARCHAR(120) NOT NULL,
  phone VARCHAR(20) NULL
);
```

`id` is MySQL's number. `employee_id` is the HR id, `E-1042`. The URL keeps using `E-1042`.

Install the database libraries in the active venv:

```
pip install sqlalchemy mysql-connector-python
```

`sqlalchemy` is the Python layer over the tables. `mysql-connector-python` is the same driver as the database programming module. The URL scheme `mysql+mysqlconnector` tells SQLAlchemy to use `mysql.connector`. Routes still call the session. They do not call `mysql.connector.connect` themselves.

The whole file is `examples/lesson05/01_mysql_read.py`.

```
import os
from sqlalchemy import Column, Integer, String, create_engine
from sqlalchemy.orm import declarative_base, sessionmaker

DATABASE_URL = os.getenv(
    "DATABASE_URL",
    "mysql+mysqlconnector://trainee:password123@localhost:3306/training_db",
)
engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False)
Base = declarative_base()

class Employee(Base):
    __tablename__ = "employee"

    id = Column(Integer, primary_key=True)
    employee_id = Column(String(10), unique=True, nullable=False)
    name = Column(String(80), nullable=False)
    designation = Column(String(80), nullable=False)
    department = Column(String(80), nullable=False)
    email = Column(String(120), nullable=False)
    phone = Column(String(20), nullable=True)
```

Name	What it is
`DATABASE_URL`	Where MySQL is. User, password, host, port, database name
`engine`	The connection machinery
`SessionLocal`	A factory. Call it to get one session for one request
`Employee`	The table, written as a Python class. This is not `EmployeeOut`

`EmployeeOut` is still the response model, with `from_attributes=True`, so a row from this class can be turned into JSON.

---

2. Insert on startup, then read

When `fastapi dev` starts, create the table if it is missing and insert the six people if the table is empty. `SEED` in the example file is that list. Ananya is the first tuple.

```
def seed(db):
    if db.query(Employee).count() > 0:
        return
    for employee_id, name, designation, department, email, phone in SEED:
        db.add(Employee(
            employee_id=employee_id,
            name=name,
            designation=designation,
```

```
            department=department,
            email=email,
            phone=phone,
        ))
    db.commit()
```

`db` is one session: `db = SessionLocal()`. `commit` writes the rows. `close` returns the connection. The example file does this from a `lifespan` function FastAPI runs at startup.

The list route asks the session, not a Python list.

```
@app.get("/employees", response_model=list[EmployeeOut])
def list_employees(department: str | None = None):
    db = SessionLocal()
    try:
        query = db.query(Employee)
        if department is not None:
            query = query.filter(Employee.department == department)
        return query.all()
    finally:
        db.close()
```

`finally` closes the session even when the query fails. The one-employee route uses the same shape and raises **404** when `one_or_none()` is `None`.

Start the app and open `GET /employees/E-1042`. Then in MySQL:

```
SELECT employee_id, name, department FROM employee;
```

You should see six rows. Restart `fastapi dev`. The six rows are still there. `seed` sees a count above zero and does not insert them again.

Common error: `Access denied for user 'trainee'`. The user or password does not match your MySQL. Set `DATABASE_URL` in the terminal before `fastapi dev`.

Common error: `Unknown database 'training_db'`. Create it: `CREATE DATABASE IF NOT EXISTS training_db;`

Common error: `Can't connect to MySQL server`. The MySQL service is not running.

Try these

1. **(Easy)** GET `/employees?department=Engineering`. You get Ananya, Karthik, and Arjun, from MySQL.
2. **(Easy)** Stop the API and run the `SELECT` above. The rows are still in the table.
3. **(Medium)** `DELETE FROM employee;` in MySQL, restart `fastapi dev`, and GET the list. The six people are back. `seed` ran because the table was empty.
4. **(Stretch)** Add a seventh person with `INSERT INTO employee ...` and GET that id without changing Python.

Interview: *What is a session?* — One conversation with MySQL for one piece of work. Open it, run the query, commit if you changed rows, close it.

Key takeaways

- `Employee` is the table. `EmployeeOut` is the JSON.
- `SessionLocal()` opens a session. `close()` finishes it.
- `commit` makes an insert stick. A restart of FastAPI does not delete the table.
- The URL still uses `E-1042`, not the numeric `id`.

Lab — Ticket ACME-205

Homework

Next: CRUD. POST, PUT, PATCH, and DELETE write through the session.